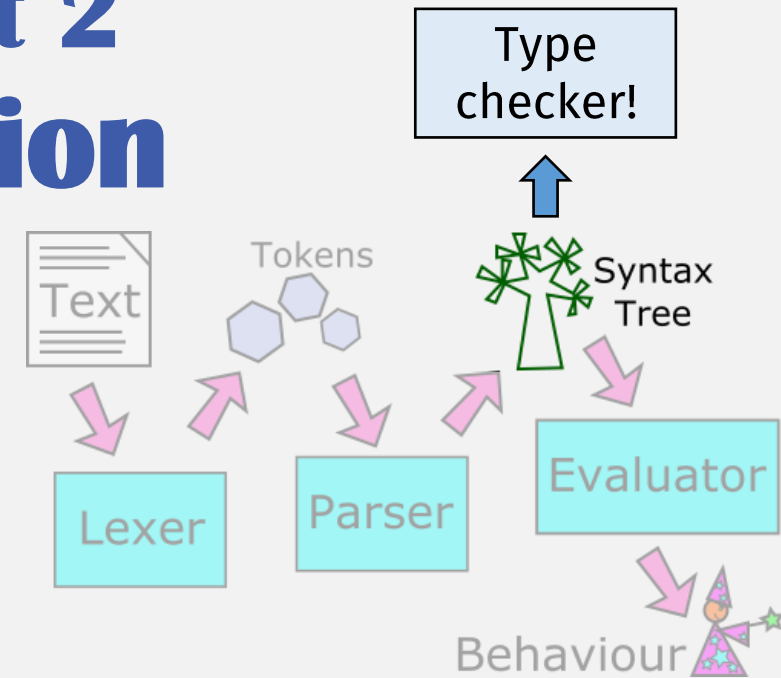


UMass Boston Computer Science
CS450 High Level Languages

Type Checking, part 2

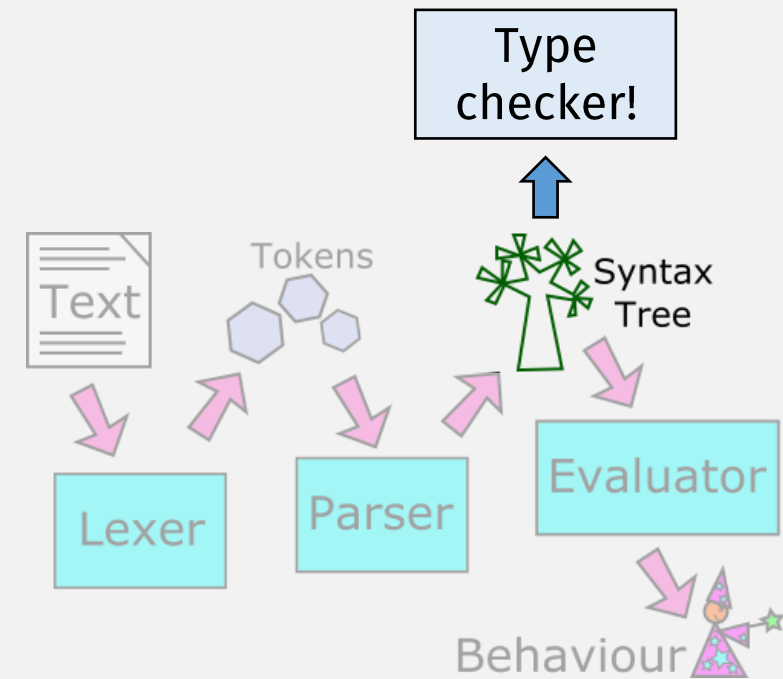
Lambda and Recursion

Tuesday, May 5, 2026



Logistics

- HW 12 out
 - due: Thurs 5/7, 11am EST



Previously

Giving Meaning to (i.e., Running) Programs

Program

(Surface Syntax)

Q₁: Can we catch errors before running a program?

A₁: Some of them!

(by “analyzing” code text)

“eval”

Result

Sometimes, the
“result” of running a
program is an Error

```
;; A Result is a:  
;; - Number  
;; - FunctionResult  
;; - ErrorResult
```

```
;; An ErrorResult is a:  
;; - undefined-var-err  
;; - not-fn-err  
;; - arity-err  
;; - circular-err
```

Parser

Program
(Surface Syntax)

Abstract Syntax
(Program) Tree (AST)

...

Target Lang n

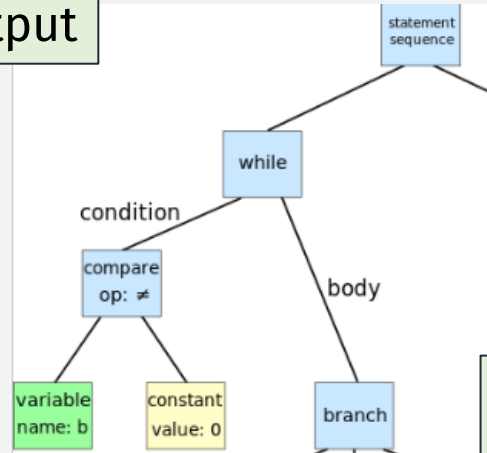
Result

input

```
(* (+ 1 2)
  (- 4 3)
  (/ 10 5))
```

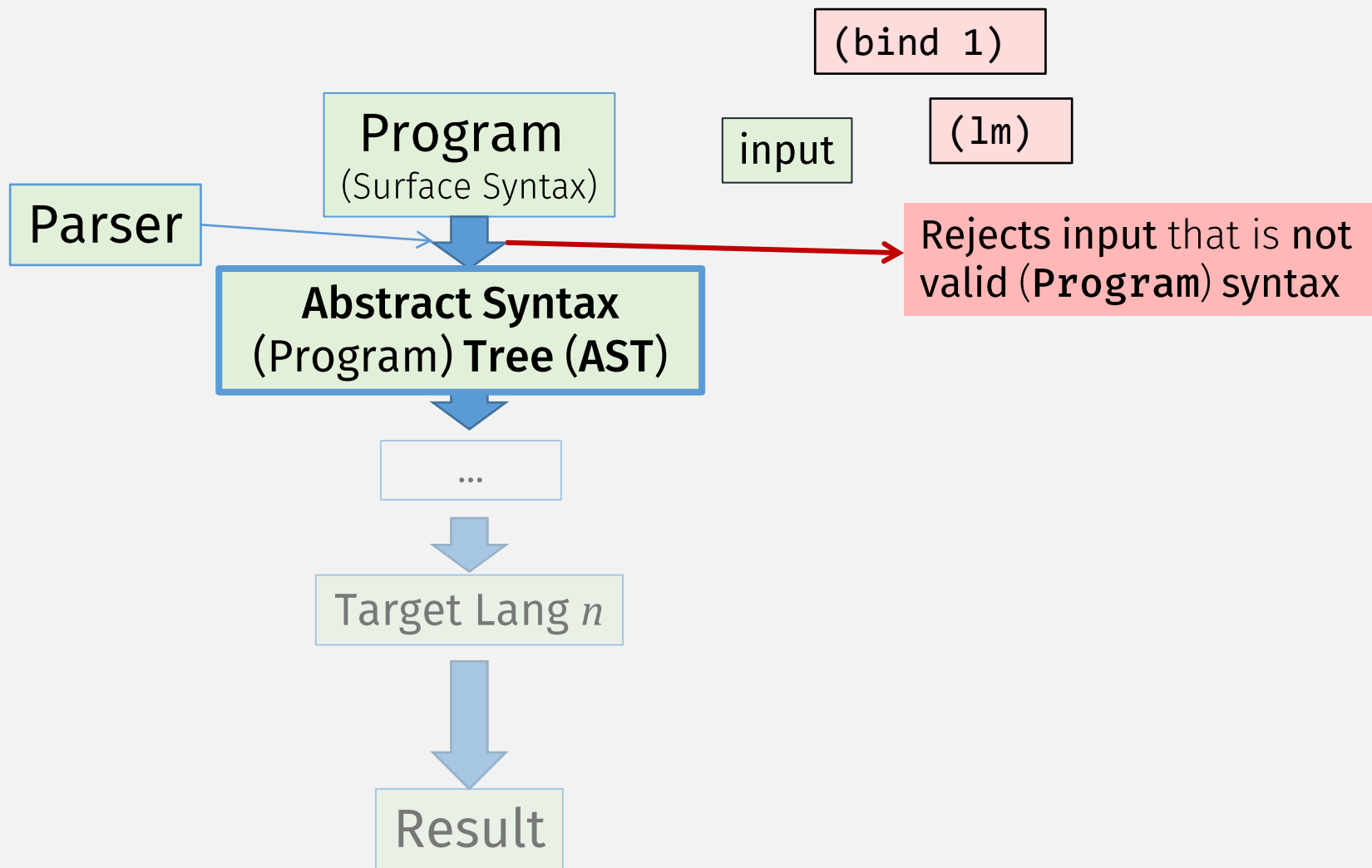
```
;; A Program (Sexpr) is a:
;; - Number
;; - `(+ ,Program ,Program)
;; - `(* ,Program ,Program)
```

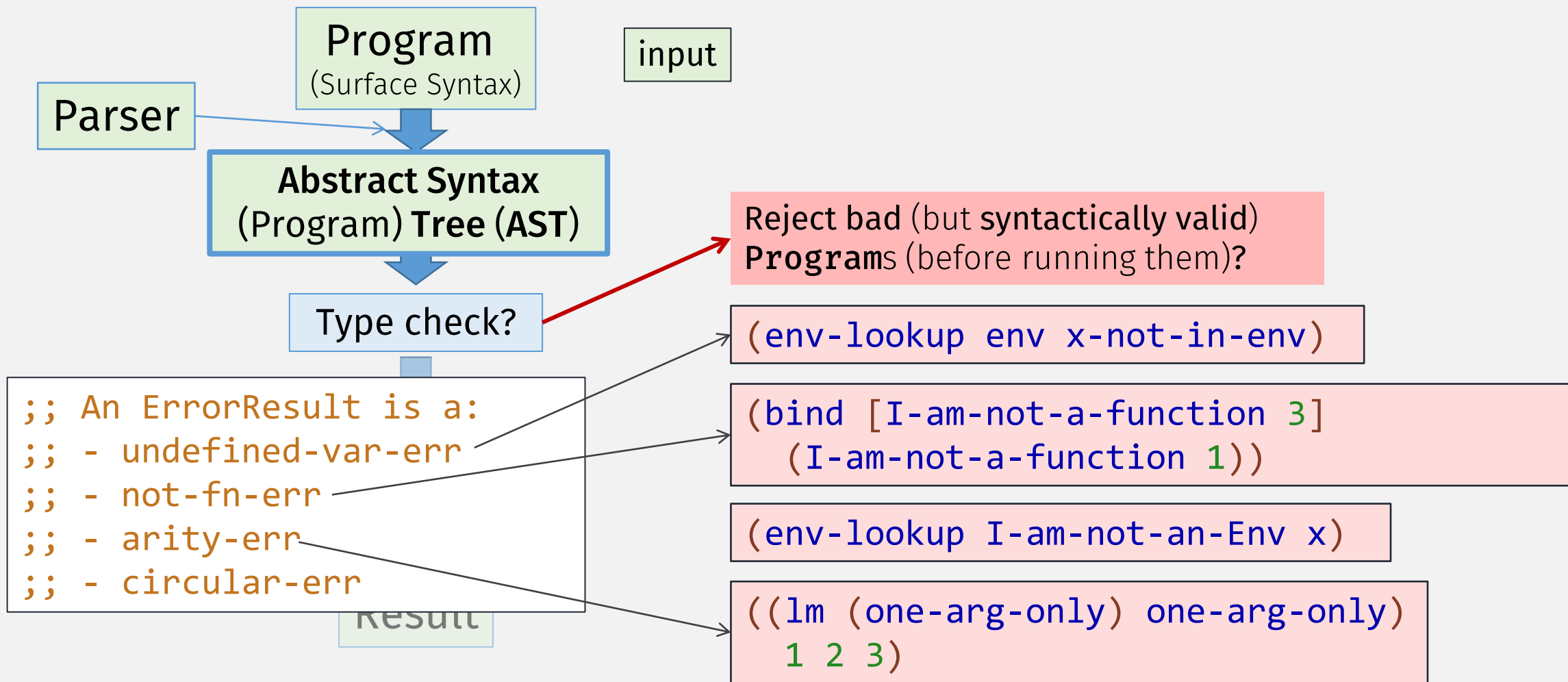
output

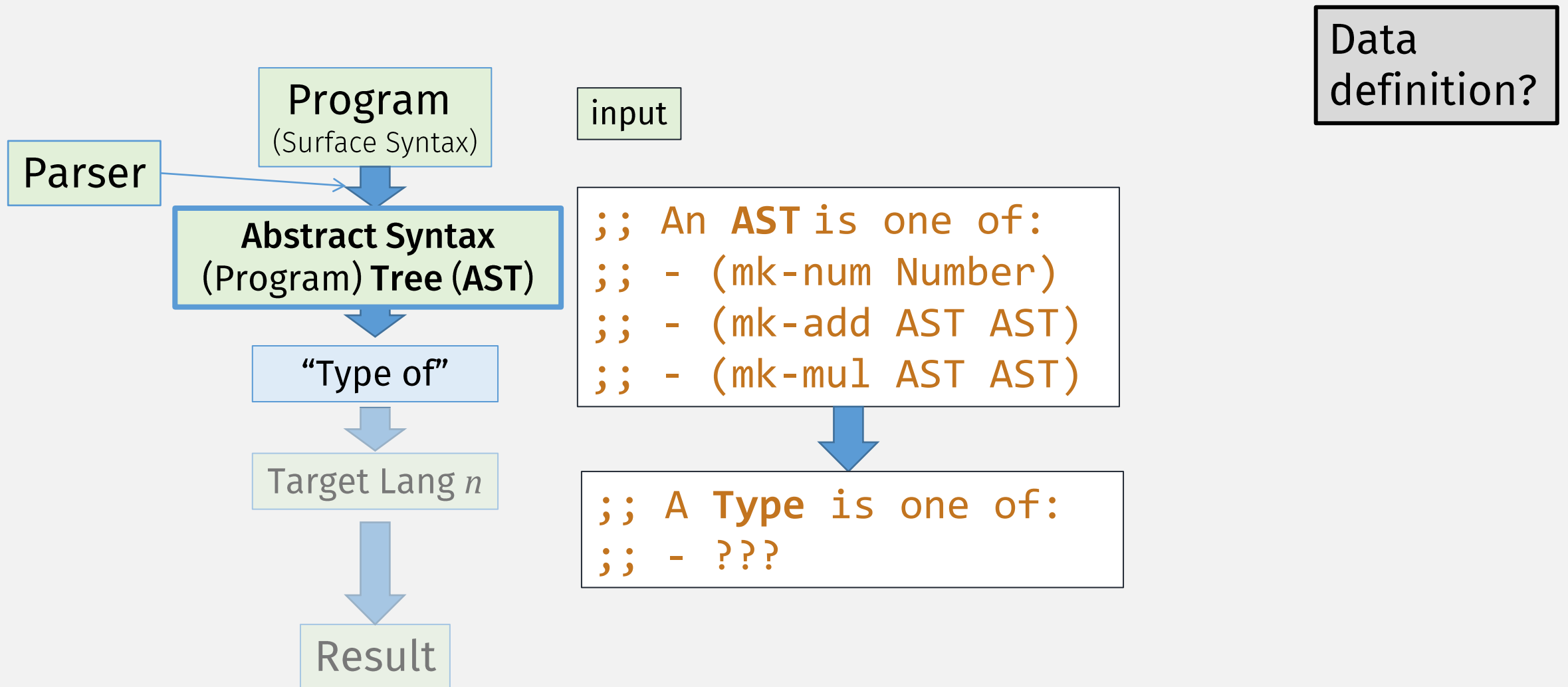


```
;; An AST is one of:
;; - (mk-num Number)
;; - (mk-add AST AST)
;; - (mk-mul AST AST)
```

tree
(grouped by features)

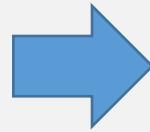






Previously

```
;; An AST is one of:  
(struct num [val])  
(struct add [lft rgt])  
(struct mul [lft rgt])
```



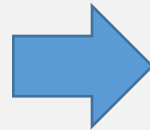
```
;; A Result is one of:  
;; - ???
```

```
;; run : AST -> Result  
;; Computes result of running given program AST
```

```
(define (run p)  
  (match p  
    [(num n) ... ]  
    [(add x y) ... (run x) ...  
                  ... (run y) ... ]  
    [(mul x y) ... (run x) ...  
                  ... (run y) ... ]))
```

TEMPLATE


```
;; An AST is one of:  
(struct num [val])  
(struct add [lft rgt])  
(struct mul [lft rgt])
```



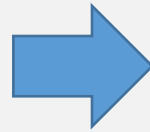
```
;; A Type is one of:  
;; - ???
```

```
;; typeof : AST -> Type  
;; Computes the Type of the given program
```

```
(define (typeof p)  
  (match p  
    [(num n) ...]  
    [(add x y) ... (typeof x) ...  
      ... (typeof y) ... ]  
    [(mul x y) ... (typeof x) ...  
      ... (typeof y) ... ]))
```

TEMPLATE

```
;; An AST is one of:  
(struct num [val])  
(struct add [lft rgt])  
(struct mul [lft rgt])
```

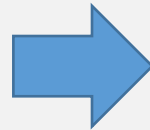


```
;; A Type is one of:  
(struct Type [])  
(struct Int Type [])
```

```
;; typeof : AST -> Type  
;; Computes the Type of the given program
```

```
(define (typeof p)  
  (match p  
    [(num n) (Int)]  
    [(add x y) ... (typeof x) ...  
                  ... (typeof y) ...]  
    [(mul x y) ... (typeof x) ...  
                  ... (typeof y) ... ]))
```

```
;; An AST is one of:  
(struct num [val])  
(struct add [lft rgt])  
(struct mul [lft rgt])
```



```
;; A Type is one of:  
(struct Type [])  
(struct Int Type [])
```

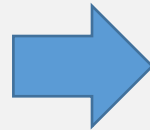
```
;; typeof : AST -> Type  
;; Computes the Type of the given program
```

```
(define (typeof p)  
  (match p  
    [(num n) (Int)]  
    [(add x y) ... (typeof x) ...  
                  ... (typeof y) ...  
                      ? (Int) ? ]  
    [(mul x y) ... (typeof x) ...  
                  ... (typeof y) ... ]
```

WHAT ABOUT TEMPLATE PIECES?

Result should be **Int** Type

```
;; An AST is one of:  
(struct num [val])  
(struct add [lft rgt])  
(struct mul [lft rgt])
```



```
;; A Type is one of:  
(struct Type [])  
(struct Int Type [])
```

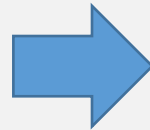
```
;; typeof : AST -> Type  
;; Computes the Type of the given program
```

```
(define (typeof p)  
  (match p  
    [(num n) (Int)]  
    [(add x y) (if (and (typeof x) (typeof y))  
                    ? (Int) ?  
                    TYPE-ERROR)])
```

(hw12)

(typeof x) (Int))
(typeof y) (Int))

```
;; An AST is one of:  
(struct num [val])  
(struct add [lft rgt])  
(struct mul [lft rgt])
```

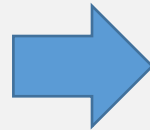


```
;; A Type is one of:  
(struct Type [])  
(struct Int Type [])  
;; or, TYPE-ERROR (???)
```

```
;; typeof : AST -> Type  
;; Computes the Type of the given program
```

```
(define (typeof p)  
  (match p  
    [(num n) (Int)]  
    [(add x y) (if (and (type=? (typeof x) (Int))  
                        (type=? (typeof y) (Int)))  
                   (Int)  
                   ??TYPE-ERROR??)]))
```

```
;; An AST is one of:  
(struct num [val])  
(struct add [lft rgt])  
(struct mul [lft rgt])
```



```
;; A Type is one of:  
(struct Type [])  
(struct Int Type [])  
;; or, TYPE-ERROR (?)
```

```
;; typeof : AST -> Type  
;; Computes the Type of the given program
```

```
(define (typeof p)  
  (match p  
    [(num n) (Int)]  
    [(add x y) (if .... (type=? (typeof x) (Int)) ....  
                        ;; else ....
```

Still sort of a “syntax time”,
i.e., static, error (because
program not run yet!)

```
(raise-syntax-error 'typeof  
  “type mismatch, expected Int, got ~a” (typeof x)  
  #:exn exn:fail:typecheck:cs450))])
```

Want: a proper (exn) type error
Because: **Program** should not be **run**

Adding Variables

Programmer writes:

```
;; An Variable is a:  
... - Symbol
```

```
;; A Program is  
;; - Atom  
;; - Variable  
;; - ...
```

Q₁: What is the **Type** of a variable?

A₁: The **Type** of the value it represents

Q₂: Where does this value come from?

A₂: Other parts of the program!

```
;; An AST is one of:
```

```
...  
;; - (mk-var Symbol)
```

```
...  
;; ...
```

Hint: Don't use "**var**"
for struct name
(reserved Racket
match pattern)

```
;; ...  
(struct vari [name])
```

```
;; A Type is one of:
```

The **typeof** function needs to "remember" these values (with an **accumulator**!)

```
(struct Int type [])  
;; - ??
```

typeof

typeof, with an accumulator

```
;; typeof: AST -> Type  
;; Computes type of a CS450 Lang Program
```

```
(define (typeof p)  
  ;; accumulator acc : TypeEnvironment  
  ;; invariant: Contains types of variables ... currently in-scope  
  (define (typeof/acc p acc)  
    (match p  
      [(num n) ....]  
      [(vari x) ....]))  
  (typeof/acc p ??? ))
```


Type Environments

- A data structure that “associates” (Var, Type) together
 - E.g., maps, hashes, etc
 - For simplicity, let’s use list-of-pairs

;; An **TypeEnvironment** is one of:

;; - empty

;; - (cons (list Var Type) TypeEnvironment)

;; Represents: a type environment for

;; cs450-lang variables

;; if there are duplicates,

;; vars at front of list shadow those in back

Type Environments

- A data structure that “associates” (Var, Type)
 - E.g., maps, hashes, etc
 - For simplicity, let’s use list-of-pairs

Reject bad (but syntactically valid) Programs (before running them)?

```
;; An ErrorResult is a:  
;; ☒ undefined-var-err  
;; - not-fn-err  
;; - arity-err  
;; - circular-err
```

```
;; An TypeEnvironment (TyEnv) is one of:  
;; - empty  
;; - (cons (list Var Type) TypeEnvironment)
```

- Needed operations:

- `tyenv-add` : TyEnv Var Type -> TyEnv
- `tyenv-lookup` : TyEnv Var -> Type

Should throw “typecheck” exn, if var not found
(don’t want to “run” the program)

typeof, with a Type Environment accumulator

```
;; typeof : AST -> Type
```

```
(define (typeof p)
  ;; accumulator tyenv : TypeEnvironment
  ;; invariant: contains in-scope Vars + their Types
  (define (typeof/env p tyenv)
    (match p
      ...
      [(vari x) (tyenv-lookup tyenv x)]
      ...
    ))
  (typeof/env p ??? ))
```

typeof, with a Type Environment accumulator

```
;; typeof : AST -> Type
```

```
(define (typeof p)
  ;; accumulator tyenv : TypeEnvironment
  ;; invariant: contains in-scope Vars + their Types
  (define (typeof/env p tyenv)
    (match p
      ...
      [(vari x) (tyenv-lookup tyenv x)]
      [(bind x e body) ... (tyenv-add tyenv x (run/env e tyenv)) ...]
      ... ))
  (typeof/env p ??? ))
```

Don't
name
this var

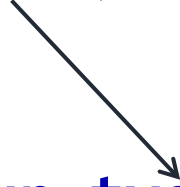
typeof, with a Type Environment accumulator

TODO:

- When are variables “added” to environment???
- What is initial environment?

```
;; typeof : AST -> Type
```

```
(define (typeof p)
  ;; accumulator tyenv : TypeEnvironment
  ;; invariant: contains in-scope Vars + their Types
  (define (typeof/env p tyenv)
    (match p
      ...
      [(vari x) (tyenv-lookup tyenv x)]
      [(bind ??? body) ... (tyenv-add tyenv x (typeof ??? /env e tyenv)) ...]
      ... ))
  (typeof/env p ??? ))
```



Programs that Add Variables to Environment

```
;; A Program is one of:  
;; - Atom  
;; - Variable (Var)  
;; - ??????
```

Programs that Add Variables to Environment

```
;; A Program is one of:  
;; - Atom  
;; - Variable (Var)  
;; - `(bind [,Var ,Program] ,Program)  
;; - ...
```

(like “**let**” in other langs)

typeof, with bind

```
;; typeof: AST -> Type
```

```
(define (typeof p)
```

```
;; accumulator tyenv : TypeEnvironment
```

```
;; invariant: contains in-scope Vars + their Types
```

```
p tyenv)
```

```
; An AST is one of:
```

```
; - ...
```

```
; - (mk-bind Symbol AST AST)
```

```
[(vari x) (tyenv-lookup tyenv x)]
```

```
[(bind x e body) ... (tyenv-add tyenv x (typeof/env e tyenv)) ...]
```

```
... ))
```

```
(typeof/env p ??? )
```

```
;; An TypeEnvironment (TyEnv) is one of:
```

```
;; - empty
```

```
;; - (cons (list Var Type) Env)
```

TypeEnvironment has **Types** (not **AST**)

How to convert **AST** to **Type**?

(From
template!)

Add to environment, bound to **x**

Be careful to get correct “**scoping**”
(x not visible in expression e,
so use unmodified input tyenv)

typeof, with **bind**

typeof must produce Type

;; typeof: AST -> Type

```
(define (typeof p)
  ;; accumulator tyenv : TypeEnvironment
  ;; invariant: contains in-scope Vars + their Types
  p tyenv)
```

; An AST is one of:

; - ...

; - (mk-bind Symbol AST **AST**)

```
  [(vari x) (tyenv-lookup tyenv x)]
  [(bind x e body) ??? (tyenv-add tyenv x (typeof/env e tyenv)) ...]
  ... ))
(typeof/env p ??? )
```

typeof, with **bind**

```
;; typeof: AST -> Type
```

```
(define (typeof p)
```

```
  ;; accumulator tyenv : TypeEnvironment
```

```
  ;; invariant: contains in-scope Vars + their Types
```

```
  (define (typeof/env p tyenv)
```

```
    (match p
```

```
      ...
```

```
      [(vari x) (tyenv-lookup tyenv x)]
```

```
      [(bind x e body) (typeof/env body (tyenv-add tyenv x (typeof/env
```

```
        ... )))
```

```
(typeof/env p ??? ))
```

(From
template!

typeof body with new env containing x

Initial Environment?

TODO:

- When are variables “added” to environment
- What is **initial environment**?

```
;; typeof: AST -> Type
```

```
(define (typeof p)
  ;; accumulator tyenv : TypeEnvironment
  ;; invariant: contains in-scope Vars + their Types
  (define (typeof/env p tyenv)
    (match p
      ...
      [(vari x) (tyenv-lookup tyenv x)]
      [(bind x e body) (typeof/env body (tyenv-add tyenv x (typeof/env
        ... )))
      ...
    ))
  (typeof/env p ??? ))
```

Initial Environment

```
;; A Program is one of:  
;; - Atom  
;; - Variable  
;; - `(bind [,Var ,Program] ,Program)  
;; - `(+,Program ,Program)  
;; - `(* ,Program ,Program)
```

These **don't** need to be separate constructs!

Put these into “initial” environment

Initial Environment

```
;; A Program is one of:  
;; - Atom  
;; - Variable  
;; - `(bind [,Var ,Program] ,Program)  
;; - `(+,Program,Program)  
;; - `(*,Program,Program)
```

Put these into “initial” environment

```
;; An TypeEnv is one of:  
;; - empty  
;; - (cons (list Var Type) TypeEnv)
```

Need new
kind of
Type

```
(define INIT-TYENV  
  `((+ ,(mk-FnTy (Int) (Int) (Int)))  
    (* ,(mk-FnTy (Int) (Int) (Int))))
```

+ variable

```
;; A Type is one of:  
(struct Type [])  
(struct Int Type [])  
;; - (mk-FnTy Type ... Type)  
(struct Fn Type [tins tout])
```

Initial Environment

But ... how to type check function applications??

```
(define INIT-TYENV `((+ ,(mk-FnTy (Int) (Int) (Int)))  
                      (* ,(mk-FnTy (Int) (Int) (Int)))))
```

```
(define (typeof p)
```

```
;; accumulator env : TypeEnvironment
```

```
(define (typeof/e p env)
```

```
  (match p
```

```
    ...
```

```
    [(vari x) (tyenv-lookup env x)]
```

```
    [(bind x e body) (typeof/e body (tyenv-add env x (typeof/e e env))
```

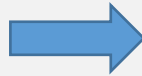
```
    ... )]
```

```
(typeof/e p INIT-TYENV))
```

Type Checking Function Application

```
;; A Program is one of:  
;; - Atom  
;; - Variable  
;; - `(bind [,Var ,Program] ,Program)  
;; - (cons Program List<Program>)
```

parse



```
;; An AST is one of:  
;; - ...  
;; - (mk-var Symbol)  
;; - (mk-bind Symbol AST AST)  
;; - (mk-appl AST List<AST>)  
  
(struct vari [name])  
(struct bind [var expr body])  
(struct appl [fn args])
```

Type Checking Function Application

TEMPLATE: extract pieces of compound data

```
(define (typeof p)
```

```
(define (tof/e p env)
  (match p
```

...

```
    [(appl fn args) (typecheck-apply
                      (tof/e fn env)
                      (map (curryr tof/e env) args))])
```

...

```
  ))
(tof/e p INIT-TYENV))
```

```
;; An AST is one of:
;; - ...
;; - (mk-var Symbol)
;; - (mk-bind Symbol AST AST)
;; - (mk-appl AST List<AST>)
```

```
(struct vari [name])
(struct bind [var expr body])
(struct appl [fn args])
```


Type Checking Function Application

```
(define (typeof p)
```

```
(define (tof/e p env)
  (match p
```

TEMPLATE: recursive calls

```
    ...
    [(appl fn args) (typecheck-apply
      (Compute type of function (tof/e fn env)
```

```
      (map (curry ??? tof/e env) args))])
```

```
    ...
  ))
  (tof/e p INIT-TYENV))
```

Compute type of args

```
;; An AST is one of:
;; - ...
;; - (mk-var Symbol)
;; - (mk-bind Symbol AST AST)
;; - (mk-appl AST List<AST>)
```

???

Type Checking Function Application

How to actually type check the application?

;; A Type is one of:
(struct Type [])
(struct Int Type [])
(struct Fn Type [tins tout])

```
(define (typeof p)
```

```
(define (tof/e p env)  
  (match p
```

...

```
    [(appl fn args) (typeof ???  
                        (tof/e fn env)  
                        (map (curryr tof/e env) args))])  
    ...
```

```
  ))  
(tof/e p INIT-TYENV))
```

Should only work for **Fn** types?

Type Checking Function Application

```
;; A Type is one of:  
(struct Type [])  
(struct Int Type [])  
(struct Fn Type [tins tout])
```

```
(define (typeof p)
```

```
  (define (tof/e p env)
```

```
    (match p
```

Example that should fail: `(typeof '(10 10)) ; apply non-fn`

```
      ...
```

```
      [(appl fn args) (typeof-apply  
                        (tof/e fn env)  
                        (map (curryr tof/e env) args))])  
      ...
```

```
    ))
```

```
(tof/e p INIT-TYENV))
```

Type Checking Function Application

```
;; A Type is one of:  
(struct Type [])  
(struct Int Type [])  
(struct Fn Type [tins tout])
```

```
(define (typeof p)
```

```
  (define (tof/e p env)  
    (match p
```

```
      ...
```

```
      [(appl fn args) (typeof-apply  
                        (tof/e fn env)  
                        (map (curryr tof/e env) args))])  
      ...
```

```
    ))  
(tof/e p INIT-TYENV))
```

Type Checking Function Application

```
;; A Type is one of:  
(struct Type [])  
(struct Int Type [])  
(struct Fn Type [tins tout])
```

Type of fn

Type of args

```
;; typeof-apply: Type Listof<Type> -> Type
```

```
(define (typeof-apply fntype argtypes)
```

```
  (match fntype
```

TEMPLATE?

```
    [(Int) ... ]
```

Type Check fail

```
    [(Fn expected-argtypes outtype) ... ]
```

```
  ))
```

Type Checking Function Application

```
;; A Type is one of:  
(struct Type [])  
(struct Int Type [])  
(struct Fn Type [tins tout])
```

```
;; typeof-apply: Type Listof<Type> -> Type
```

```
(define (typeof-apply fntype argtypes)  
  (match fntype  
    [(Fn expected-argtypes outtype)  
     (if  
       (andmap type=? expected-argtypes argtypes)  
       outtype  
       (raise-type-exn ....))])))
```

Check correct number of arguments???

Result is
fn output
type

If input arg types match the expected ...

Type Checking Function Application

Reject bad (syntactically valid)
Programs (before running them)?

```
;; An ErrorResult is a:  
;; ☒ undefined-var-err  
;; ☒ not-fn-err  
;; ☒ arity-err  
;; - circular-err
```

```
;; A Type is one of:  
(struct Type [])  
(struct Int Type [])  
(struct Fn Type [tins tout])
```

```
;; typeof-apply: Type Listof<Type> -> Type
```

```
(define (typeof-apply fntype argtypes)  
  (match fntype  
    [(Fn expected-argtypes outtype)   
      (if (and (= (length expected-argtypes) (length argtypes))  
                (andmap type=? expected-argtypes argtypes))  
          outtype  
          (raise-type-exn ....) )]))
```

Check correct number of arguments???

Type Checking CS450 Lang

```
;; A Program is one of:  
;; - Atom  
;; - Variable  
;; - `(bind [,Var ,Program] ,Program)  
;; - (cons Program List<Program>)
```

What if a user wants to define their own function!

Next Feature: Lambdas?

“Lambdas” in CS450 Lang

```
;; A Program is one of:  
;; - Atom  
;; - Variable (Var)  
;; - `(bind [,Var ,Program] ,Program)  
;; - `(lm ,List<Var> ,Program)  
;; - (cons Program List<Program>)
```

CS450 Lang “Lambda” examples

CS450LANG

(lm (x) (+ x 1))

Type of x? Int

;; A Program is one of:

;; - Atom

;; - Variable (Var)

;; - `(bind [,Var ,Program] ,Program)

;; - `(lm ,List<Var> ,Program)

;; - (cons Program List<Program>)

(lm (x) x)

Type of x? ???

Programmer needs to give it!

Typed CS450 Lang “Typed Lambda” examples

CS450LANG

(**lm** [**x** : Int] (+ **x** **1**))

Type of x? Int

```
;; A Program is one of:  
;; - Atom  
;; - Variable (Var)  
;; - `(bind [,Var ,Program] ,Program)  
;; - `(lm [,Var : ,Type] ,Program)  
;; - (cons Program List<Program>)
```

(**lm** [**x** : Int] **x**)

Type of x? Int

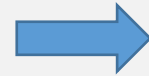
(single-arity for now)

Programmer needs to give it!

Typed CS450 Lang – parsing “Typed Lambda”

```
;; A Program is one of:  
;; - Atom  
;; - Variable (Var)  
;; - `(bind [,Var ,Program] ,Program)  
;; - `(lm [,Var : ,Type] ,Program)  
;; - (cons Program List<Program>)
```

parse



```
;; An AST is one of:  
;; ...  
;; - (mk-tylm-ast Var Type AST)  
;; ...  
  
(struct tylm-ast [x ty body])  
;; ...
```

Parsing “Typed Lambda”

TYPED CS450LANG

(lm [x : Int] (+ x 1))

```
(define/contract (parse p)
  (-> Program? AST?)
  (match p
```

```
    ...
    [ `(lm [ , (? symbol? x) : ,ty] ,bod) ... ]
```

Pattern matching
correct typed lm syntax

```
    [ `( ,fn . ,args) ... ]
    [ _ (raise-syntax-error
          'parse "not a valid CS450 Lang program" p
          #:exn exn:fail:syntax:cs450) ] )
```

Parsing “Typed Lambda”

TYPED CS450LANG

```
(lm [x : Int] (+ x 1))
```

```
(define/contract (parse p)
  (-> Program? AST?)
  (match p
```

...

```
    [`(lm [, (? symbol? x) : ,ty] ,bod)
     (mk-tylm-AST x (parse-type ty) (parse bod)) ]
```

not an AST!

```
    [`(,fn . ,args) ... ]
    [_ (raise-syntax-error
        'parse "not a valid CS450 Lang program" p
        #:exn exn:fail:syntax:cs450))]])
```

;; An AST is one of:

;; ...

;; - (mk-lm-ast Var Type AST)

;; ...

Recursive call for ASTs

Parsing “Type Syntax”? (what the programmer writes)

```
;; TypeSyntax is:  
;; - 'Int
```



```
;; A Type is one of:  
;; (mk-Int)  
;; (mk-FnTy Type ... Type)
```

```
(define/contract (parse-type ty)  
  (-> TypeSyntax? Type?)  
  (match ty  
    ...  
    ['Int ... ]  
  
    ))
```

Parsing “Type Syntax”? (what the programmer writes)

```
;; TypeSyntax is:  
;; - 'Int
```



```
;; A Type is one of:  
;; (mk-Int)  
;; (mk-FnTy Type ... Type)
```

```
(define/contract (parse-type ty)  
  (-> TypeSyntax? Type?)  
  (match ty  
    ...  
    ['Int (mk-Int)]
```

```
))
```


Parsing “Type Syntax”? (what the programmer writes)

```
;; TypeSyntax is:  
;; - 'Int  
;; - '(-> TypeSyntax ...)
```



```
;; A Type is one of:  
;; (mk-Int)  
;; (mk-FnTy Type ... Type)
```

```
(define/contract (parse-type ty)  
  (-> TypeSyntax? Type?)  
  (match ty  
    ...  
    ['Int (mk-Int)]
```

```
))
```

Parsing “Type Syntax”? (what the programmer writes)

```
;; TypeSyntax is:  
;; - 'Int  
;; - '(-> TypeSyntax ...)
```



```
;; A Type is one of:  
;; (mk-Int)  
;; (mk-FnTy Type ... Type)
```

```
(define/contract (parse-type ty)  
  (-> TypeSyntax? Type?)  
  (match ty  
    ...  
    ['Int (mk-Int)]  
    [`(-> ,tyins ... ,tyout) ... ]
```

Must have **output type**, and “zero or more” input types

))

Parsing “Type Syntax”? (what the programmer writes)

```
;; TypeSyntax is:  
;; - 'Int  
;; - '(-> TypeSyntax ...)
```



```
;; A Type is one of:  
;; (mk-Int)  
;; (mk-FnTy Type ... Type)
```

```
(define/contract (parse-type ty)  
  (-> TypeSyntax? Type?)  
  (match ty  
    ...  
    ['Int (mk-Int)]  
    [`(-> ,tyins ... ,tyout) (mk-FnTy tyins tyout)]
```

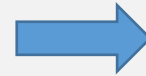
```
))
```

Typed CS450 Lang – type of Typed Lambda

```
;; A Program is one of:  
;; - Atom  
;; - Variable (Var)  
;; - `(bind [,Var ,Program] ,Program)  
;; - `(lm [,Var : ,Type] ,Program)  
;; - (cons Program List<Program>)
```

```
;; - ???
```

parse



typeof



```
;; An AST is one of:  
;; ...  
;; - (mk-tylm-ast Var Type AST)  
;; ...
```

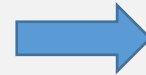
```
(struct tylm-ast [x ty body])  
;; ...
```

Typed CS450 Lang – type of Typed Lambda

```
;; A Program is one of:  
;; - Atom  
;; - Variable (Var)  
;; - `(bind [,Var ,Program] ,Program)  
;; - `(lm [,Var : ,Type] ,Program)  
;; - (cons Program List<Program>)
```

```
;; A Type is one of:  
;; (mk-Int)  
;; (mk-FnTy Type ... Type)
```

parse



```
;; An AST is one of:  
;; ...  
;; - (mk-tylm-ast Var Type AST)  
;; ...
```

```
(struct tylm-ast [x ty body])  
;; ...
```

typeof



Typeof Lambdas

```
;; typeof: AST -> Type
```

```
(define (typeof p)
```

TEMPLATE?

```
(define (tof/e p env)
```

```
(match p
```

...

```
[(tylm-ast x ty body) ??           ??           ??]
```

...

```
))
```

```
(tof/e p INIT-TYENV))
```

```
;; An AST is one of:
```

```
;; ...
```

```
;; - (mk-tylm-ast Var Type AST)
```

```
;; ...
```

```
(struct tylm-ast [x ty body])
```

Typeof Lambdas

```
;; typeof: AST -> Type
```

```
(define (typeof p)
```

TEMPLATE

```
(define (tof/e p env)
```

```
(match p
```

...

```
[(tylm-ast x ty body) ?? x ty ?? (tof/e body env) ??]
```

...

```
))
```

```
(tof/e p INIT-TYENV))
```

```
;; An AST is one of:
```

```
;; ...
```

```
;; - (mk-tylm-ast Var Type AST)
```

```
;; ...
```

```
(struct tylm-ast [x ty body])
```

Typeof Lambdas

```
;; typeof: AST -> Type
```

```
(define (typeof p)
```

```
  (define (tof/e p env)
    (match p
```

...

```
      [(tylm-ast x ty body) ?? x ty ?? (tof/e body env) ??]
```

What is the Type of a **tylm** function?

```
    ))
```

```
(tof/e p INIT-TYENV))
```

(-> "input type" "output type")

given

Must be computed

```
;; An AST is one of:
;; ...
;; - (mk-tylm-ast Var Type AST)
;; ...
(struct tylm-ast [x ty body])
```

```
;; A Type is one of:
;; (mk-Int)
;; (mk-FnTy Type ... Type)
```


Typeof Lambdas

```
;; typeof: AST -> Type
```

```
(define (typeof p)
```

```
  (define (tof/e p env)
```

```
    (match p
```

```
      ...
```

Type of x is function input type

```
      [(tylm-ast x ty body) (mk-FnTy ty (tof/e body (env-add env x ty)))
```

```
      ...
```

Type of body is function output type

Can't compute unless x is in type env!

```
    ))
```

```
  (tof/e p INIT-TYENV))
```

(-> "input type" "output type")

given

Must be computed

```
;; A Type is one of:
```

```
;; (mk-Int)
```

```
;; (mk-FnTy Type ... Type)
```

Type Checking Function Application

```
;; A Type is one of:  
(struct Type [])  
(struct Int Type [])  
(struct Fn Type [tins tout])
```

Confirm this also works for applying typed `lm`!

```
;; typeof-apply: Type Listof<Type> -> Type
```

```
(define (typeof-apply fntype argtypes)  
  (match fntype  
    [(Fn expected-argtypes outtype)  
     (if (and (= (length expected-argtypes) (length argtypes))  
               (andmap type=? expected-argtypes argtypes))  
         outtype  
         (raise-type-exn ....) )])])
```

Next: Recursion with Types?

Previously

bind in CS450 Lang

;; A Variable (Var) is a Symbol

;; A Prog is one of:

;; ...

;; - Var

;; - `(bind [,Var ,Prog] ,Prog)

;; ...

Reference a variable binding

new binding is in-scope
(can be referenced) here

Create a new
variable binding

new binding is **not**
in-scope here



Previously

bind examples

```
;; A Prog is one of:  
;; ...  
;; - Var  
;; - `(bind [,Var ,Prog] ,Prog)  
;; ...
```



new binding is **not**
in-scope here

```
(check-equal?  
  (eval450  
    '(bind [x (+ x 20)]  
           x))  
  UNDEFINED-ERROR )
```

???

bind examples, with functions

```
;; A Prog is one of:
```

```
;; ...
```

```
;; - Var
```

```
;; - `(bind [,Var ,Prog] ,Prog)
```

```
    `(lm ,List<Var> ,Prog)
```

```
    (cons Prog List<Prog>)
```

```
;; ...
```

"lambda"
function

function call

function

arguments

```
(check-equal?
```

```
  (eval450
```

```
    '(bind [f (lm (x) (+ x 4))]
```

```
          (f 6)))
```

```
  10 )
```

f not in-scope here
(so function can't be recursive!)

```
(check-equal?
```

```
  (eval450
```

```
    '(bind [f (lm (x) (f x))]
```

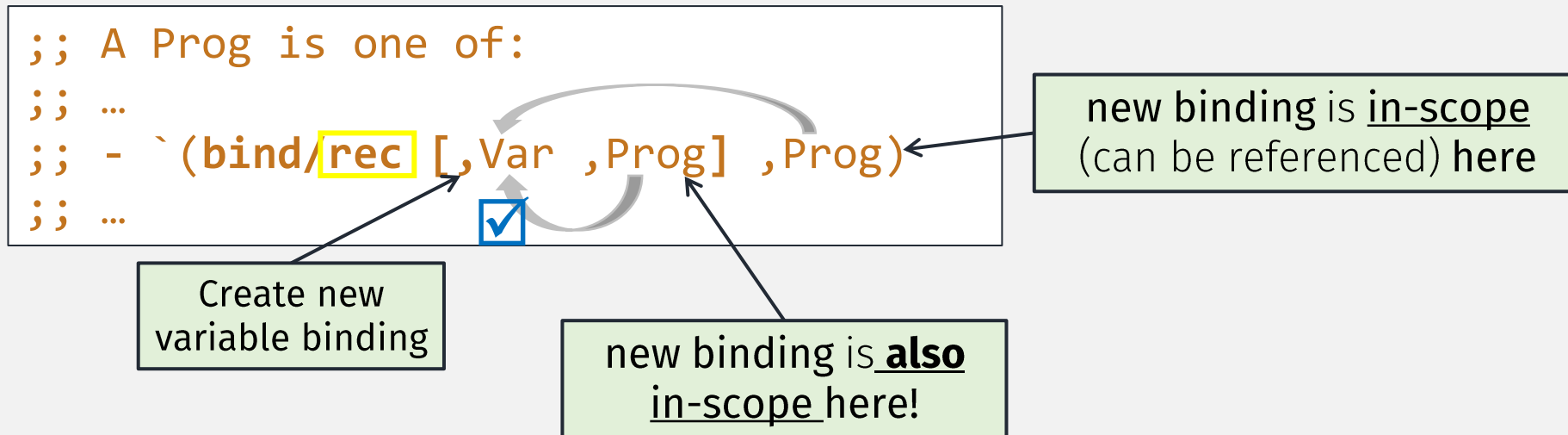
```
          (f 6)))
```

```
  UNDEF-ERR)
```

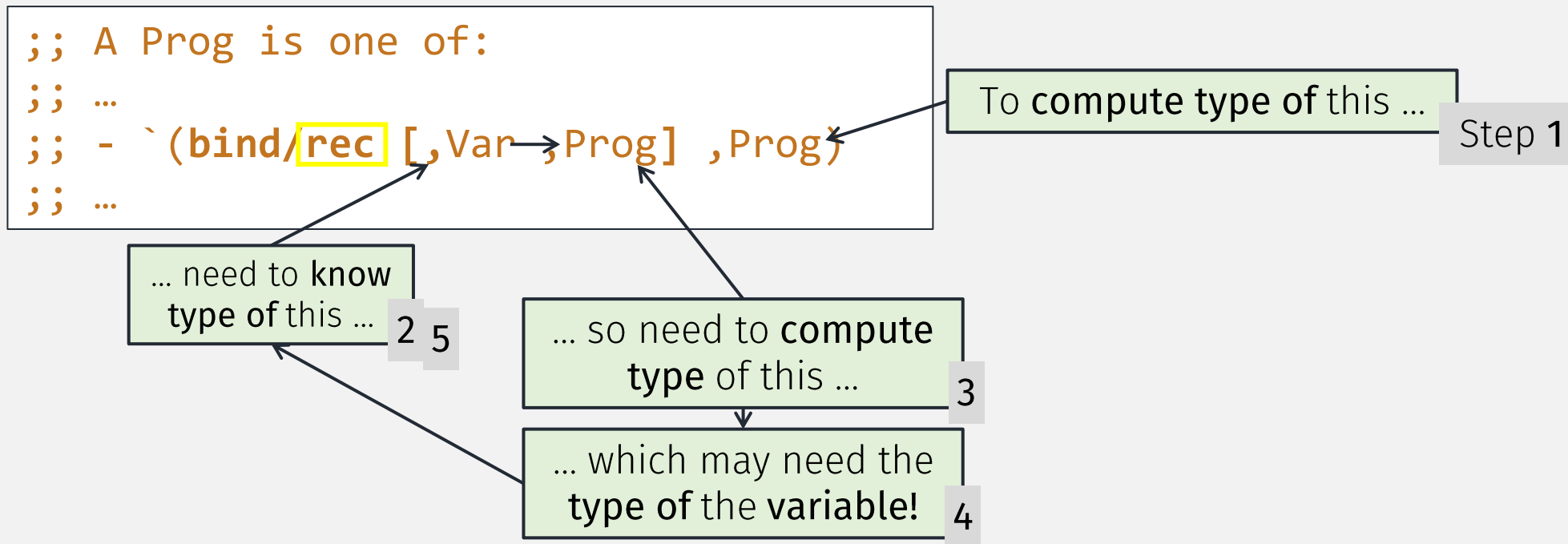
f not in-scope here
(so function can't be recursive!)

How to add recursion to our language?

“bind/rec” in CS450 Lang



“bind/rec” in Typed CS450 Lang



How to resolve this circularity??

bind/rec examples

```
;; A Prog is one of:  
;; ...  
;; - `(bind/rec [ ,Var ,Prog] ,Prog)  
;; - `(iffy ,Prog ,Prog ,Prog)  
;; ...
```

... need to know type of
this (function name) ...

... so need to compute
type of this (function) ...

CS450LANG

... which may need the
type of the variable!

```
(bind/rec  
[fac  
  (lm (n)  
    (iffy n  
      (* n (fac (- n 1)))  
      1))]  
(fac 5)) ; => 120
```

How to resolve this circularity??

To compute type of this ...

bind/rec examples - running

```
;; A Prog is one of:  
;; ...  
;; - `(bind/rec [ ,Var ,Prog] ,Prog)  
;; - `(iffy ,Prog ,Prog ,Prog)  
;; ...
```

When **running**,
circularity resolved by
first adding a
“placeholder” for this

... which “works” because the “
lm result” delays running the
body until it is applied ...

CS450LANG

```
(bind/rec  
[fac  
 (lm (n)  
   (iffy n  
    (* n (fac (- n 1)))  
    1))]  
(fac 5)) ; => 120
```

How to resolve this circularity??

bind/rec examples – type checking

```
;; A Prog is one of:  
;; ...  
;; - `(bind/rec [ ,Var ,Prog] ,Prog)  
;; - `(iffy ,Prog ,Prog ,Prog)  
;; ...
```

type checking cannot
resolve circularity in
the same way ...

... because to compute
typeof the function ...

CS450LANG

```
(bind/rec  
[fac  
 (lm (n)  
   (iffy n  
        (* n (fac (- n 1)))  
        1))]  
(fac 5)) ; => 120
```

... we need the typeof
the body now!

How to resolve this circularity??

“bind/rec” in Typed CS450 Lang

```
;; A Prog is one of:  
;; ...  
;; - `(bind/rec [ ,Var : ,Type ,Prog] ,Prog)  
;; ...
```

```
(bind/rec  
  [fac : (-> Int Int)  
    (lm [x : Int]  
      (iffy n  
        (* n (fac (- n 1)))  
        1)))]  
  (fac 5)) ; => 120
```

Note: must know the
fn output type too

Note: only knowing fn
input type is not enough

How to resolve this circularity??

Require programmer to give the type!

Typeof bind/rec

TEMPLATE : extract pieces

```
;; typeof: AST -> Type
```

```
(define (typeof p)
  (define (tof/env p env)
    (match p
      ...
      [(tyrecb f tyf-given maybe-lm body)
       (define env/f (tyenv-add env f tyf-given))
       (define tyf (tof/env maybe-lm env/f))
       (if (type=? tyf tyf-given)
           (tof/env body env/f)
           (raise-type-error ....))]
      ... ))
  (tof/env p INIT-TYENV ))
```

;; An AST is one of:

;; ...

;; - (mk-**tyrecb** Symbol Type AST AST)

;; ...

(struct **tyrecb** [**var ty e body**])

Typeof bind/rec

```
;; typeof: AST -> Type
```

```
(define (typeof p)
  (define (tof/env p env)
    (match p
      ...
      [(tyrecb f tyf-given maybe-lm body)
       (define env/f (tyenv-add env f tyf-given))
       (define tyf (tof/env maybe-lm env/f))
       (if (type=? tyf tyf-given)
           (tof/env body env/f)
           (raise-type-error ....))]
      ... ))
  (tof/env p INIT-TYENV ))
```

```
;; An AST is one of:
```

```
;; ...
```

```
;; - (mk-tyrecb Symbol Type AST AST)
```

```
;; ...
```

```
(struct tyrecb [var ty e body])
```

Add **Var** and its **Type** to TyEnv

Typeof bind/rec

Template: recursive reference

```
;; typeof: AST -> Type
```

```
(define (typeof p)
  (define (tof/env p env)
    (match p
      ...
      [(tyrecb f tyf-given maybe-lm body)
       (define env/f (tyenv-add env f tyf-given))
       (define tyf (tof/env maybe-lm env/f))
       (if (type=? tyf tyf-given)
           (tof/env body env/f)
           (raise-type-error ....))]
      ... ))
  (tof/env p INIT-TYENV ))
```

```
;; An AST is one of:
;; ...
;; - (mk-tyrecb Symbol Type AST AST)
;; ...
(struct tyrecb [var ty e body])
```

Typeof bind/rec

Template: recursive reference

```
;; typeof: AST -> Type
```

```
(define (typeof p)
  (define (tof/env p env)
    (match p
      ...
      [(tyrecb f tyf-given maybe-lm body)
       (define env/f (tyenv-add env f tyf-given))
       (define tyf (tof/env maybe-lm env/f))
       (if (type=? tyf tyf-given)
           (tof/env body env/f)
           (raise-type-error ....))]
      ... ))
  (tof/env p INIT-TYENV ))
```

;; An AST is one of:

;; ...

;; - (mk-tyrecb Symbol Type AST **AST**)

;; ...

(struct tyrecb [var ty e **body**])

Typeof bind/rec

```
;; typeof: AST -> Type
```

```
(define (typeof p)
  (define (tof/env p env)
    (match p
      ...
      [(tyrecb f tyf-given maybe-lm body)
       (define env/f (tyenv-add env f tyf-given))
       (define tyf (tof/env maybe-lm env/f))
       (if (type=? tyf tyf-given)
           (tof/env body env/f)
           (raise-type-error ....))]
      ... ))
  (tof/env p INIT-TYENV ))
```

```
;; An AST is one of:
```

```
;; ...
```

```
;; - (mk-tyrecb Symbol Type AST AST)
```

```
;; ...
```

```
(struct tyrecb [var ty e body])
```

Must check computed type matches expected ...

Typeof bind/rec

```
;; typeof: AST -> Type
```

```
(define (typeof p)
  (define (tof/env p env)
    (match p
      ...
      [(tyrecb f tyf-given maybe-lm body)
       (define env/f (tyenv-add env f tyf-given))
       (define tyf (tof/env maybe-lm env/f))
       (if (type=? tyf tyf-given)
           (tof/env body env/f)
           (raise-type-error ....))]
      ... ))
  (tof/env p INIT-TYENV ))
```

```
(bind/rec
 [fac : (-> Int Int)
 typeof (lm [x : Int]
            (iffy n
                (* n (fac (- n 1)))))
 1))] ; (-> Int Int)
(fac 5)) ; => 120
```



Must check computed type matches expected ...

Reject bad (syntactically valid)
Programs (before running them)?

```
;; An ErrorResult is a:  
;; ☒ undefined-var-err  
;; ☒ not-fn-err  
;; ☒ arity-err  
;; ? circular-err ??
```

So ... can type checking catch this runtime error?

Possible, but requires “more than” type checking – a deeper “static analysis”

So ... can type checking catch all runtime errors? ...

Errors Not Caught by Type Checker

- Errors involving specific values

- E.g., divide by 0
- “off by 1”

```
(define (maybe-div0 y x) (/ y x))
```

- “Logical” errors

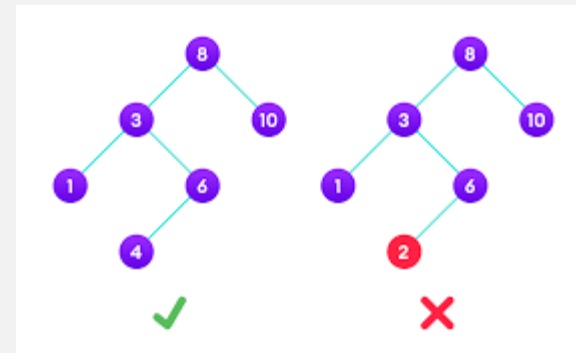
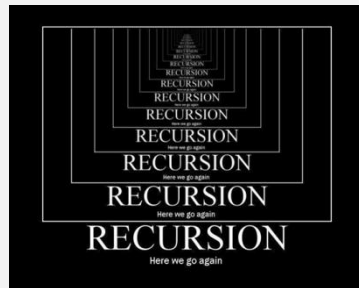
- i.e., “wrong” answers
- i.e., code that doesn’t follow purpose stmt!

```
(define (add2 x) (+ 3 x))
```

- Invariant violations

- E.g., binary search tree, red-black tree

- Infinite Loops!



- see “Halting problem” (take CS 420!)

Limits of Type Checking

- Should be **relatively fast to compute** (and **terminate!**)
 - Can only approximate runtime behavior
 - Often rejects some valid programs

```
(+ 10 (iffy 0 "one" 1))
```

- Can have steep learning curve
 - E.g., Rust

Why is Rust difficult?

Rust is considered difficult to learn by many people. Indeed, when I learned it, I considered the hardest programming language up to that time I've met. And that says something, sir

- Might not support full “type inference”
 - Can require some programmer annotations
 - Or a lot!, see Rust, Lean (“dependent” types)